Deep Dive: Java Objects, Memory, and Parameter Passing

Let me break down every single concept from this lecture in extreme depth.

---

1. 🏗️ What Does an Object Actually Contain Internally?

Setting the Stage

When you write:

```
Student s1 = new Student("Alice", 20, 101, "MIT");
```

Most beginners think `s1` **is** the object. It is not. There are **two completely separate things** happening here:

1. **The object** — lives in **Heap memory**
2. **The reference variable `s1`** — lives in **Stack memory**

Let's understand both deeply.

```
### Stack vs Heap — The Foundation
```

```
┌──────────────────────┐        ┌──────────────────────────────┐
│    STACK MEMORY      │        │         HEAP MEMORY          │
│                      │        │                              │
│  s1 → [address:1001]─────────→│  [Student Object @ 1001]     │
│                      │        │   name    → [ref to String]  │
│  (local variables,   │        │   age     → 20               │
│   method calls,      │        │   rollNo  → 101              │
│   references)        │        │   college → [ref to String]  │
└──────────────────────┘        └──────────────────────────────┘
```

- **Stack** is fast, small, automatically managed. Stores local variables and references.
- **Heap** is large, dynamic. Every `new` keyword allocates space here.

Reference Variable Size

`s1` is stored in Stack. How much space does it take?

JVM Type	Size
64-bit JVM **with** Compressed OOPs (default)	**4 bytes**
64-bit JVM **without** Compressed OOPs	**8 bytes**

> **"Compressed OOPs"** = Compressed Ordinary Object Pointers. By default, modern 64-bit JVMs compress heap addresses to fit in 4 bytes (works for heaps up to 32GB). This is why the default is 4 bytes.

So `s1` takes **4 bytes** in stack. That's it. Just a 4-byte address pointing somewhere in heap.

---

2. 📦 The Real Internal Structure of an Object

Now let's open up the actual object in heap. An object is NOT just your fields. It has **three parts**:

```
┌──────────────────────────────────────────┐
│              OBJECT IN HEAP              │
├──────────────────────────────────────────┤
│  PART 1: Object Header                   │
│    ├── Mark Word        (8 bytes)        │
│    └── Class Pointer    (4 bytes)        │
├──────────────────────────────────────────┤
│  PART 2: Instance Fields (your data)     │
│    ├── name     (String ref = 4 bytes)   │
│    ├── age      (int = 4 bytes)          │
│    ├── rollNo   (int = 4 bytes)          │
│    └── college (String ref = 4 bytes)    │
├──────────────────────────────────────────┤
│  PART 3: Padding (alignment bytes)       │
│    └── extra bytes to reach multiple     │
│        of 8                              │
└──────────────────────────────────────────┘
```

PART 1: Object Header — Deep Explanation

The header is metadata **about** the object. Think of it like a passport — it doesn't contain your belongings, but describes you.

A) Mark Word (8 bytes)

This single 8-byte block stores multiple things packed together. The JVM reuses these bits for different purposes depending on the object's current state:

Mark Word (64 bits) contains:

```
├── Identity HashCode
│   The result of System.identityHashCode(obj)
│   Stored here once computed
│
├── Locking Information
│   Is this object locked? By which thread?
│   Used in synchronized blocks:
│   synchronized(obj) { ... }
│   ─ Biased Lock bits
│   ─ Lightweight Lock bits
│   ─ Heavyweight (Monitor) Lock bits
│
├── GC Age (Garbage Collection Age)
│   How many GC cycles has this object survived?
│   Young generation objects start at age 0
│   Every GC survival increments this
│   At age 15 (default), object moves to Old Gen
│
└── GC Mark bits
    Used by GC to mark objects as reachable
    during mark-sweep cycles
```

You never set these manually — JVM manages them internally.

B) Class Pointer (4 bytes with compressed OOPs)

This is a reference to the **Class object** of this instance. Think of it as: *"What blueprint was I made from?"*

```
Student Object
│
└── classPointer ──► Student.class (in Method Area/Metaspace)
                    ├── method bytecodes
                    ├── static variables
                    └── class-level metadata
```

This is how JVM knows at runtime that `s1` is a `Student` object and not a `Person` object. This also powers things like `instanceof`, reflection, and polymorphism.

> **This is also related to the `this` keyword!** Every object knows which class it belongs to through this class pointer.

PART 2: Instance Fields — Your Actual Data

These are the variables you declared inside the class. Each field takes space based on its type:

```
class Student {
   String name;    // reference → 4 bytes
   int age;        // primitive → 4 bytes
   int rollNo;     // primitive → 4 bytes
   String college; // reference → 4 bytes
}
```

Total field size = 4 + 4 + 4 + 4 = 16 bytes

Important nuance about String fields:

`name` and `college` are **references**, not the actual string data. They store a 4-byte address pointing to a String object elsewhere in the heap.

```
Student Object                          Heap (elsewhere)
├── name ──────────────────────────► String Object ("Alice")
├── age = 20
├── rollNo = 101
└── college ────────────────────────► String Object ("MIT")
```

PART 3: Padding — The Alignment Trick

Modern CPUs don't read memory byte by byte — they read in **chunks of 8 bytes** at a time (on 64-bit systems). This is called a **word** or **cache line alignment**.

To make memory access efficient, the JVM ensures every object's total size is a **multiple of 8**.

Why multiple of 8?

Imagine the CPU reads from address 0 to 7, then 8 to 15, then 16 to 23, etc. If your object starts at address 0 and is exactly 28 bytes, the CPU needs **4 reads** — and the last read is partial, wasteful. But if it's padded to 32 bytes, it fits neatly in **4 reads** with no waste.

How padding is calculated:

```
Total = Header + Fields
      = 12 + 16
```

= 28 bytes

Next multiple of 8 after 28 = 32

Padding needed = 32 - 28 = 4 bytes

```
### Full Object Size Calculation — Student
```

```
┌──────────────────────────────────────┐
│  Mark Word            →  8 bytes     │
│  Class Pointer        →  4 bytes     │
│  ──────────────────────────────      │
│  Header Total         → 12 bytes     │
│                                      │
│  name (String ref)    →  4 bytes     │
│  age (int)            →  4 bytes     │
│  rollNo (int)         →  4 bytes     │
│  college (String ref)→  4 bytes      │
│  ──────────────────────────────      │
│  Fields Total         → 16 bytes     │
│                                      │
│  Header + Fields      → 28 bytes     │
│  Padding              →  4 bytes     │
│  ──────────────────────────────      │
│  FINAL OBJECT SIZE    → 32 bytes ✅  │
└──────────────────────────────────────┘
```

Full Object Size Calculation — Person (with byte field)

```
class Person {
    byte age; // 1 byte
}
```

Mark Word → 8 bytes
Class Pointer → 4 bytes
Header → 12 bytes

age (byte) → 1 byte
Fields → 1 byte

Total so far → 13 bytes

Next multiple of 8 = 16
Padding needed = 16 - 13 = 3 bytes

FINAL SIZE = 16 bytes ✅

---

3. 📞 Call by Value — Deep Explanation

What Happens When You Pass a Primitive to a Method?

```
class Demo {
    public static void main(String[] args) {
        int x = 4;
        int y = 5;

        System.out.println(x + " " + y); // prints: 4 5

        add10(x, y);

        System.out.println(x + " " + y); // still prints: 4 5 !!
    }

    static void add10(int x, int y) {
        x = x + 10;
        y = y + 10;
    }
}
```

Beginners expect `14 15` in the second print. They get `4 5`. Why?

Memory Walkthrough — Step by Step

Step 1: main() starts, local variables created

```
**Step 1: main() starts, local variables created**
```

```
STACK

┌──────────────────┐
│  main()          │
│  x = 4           │
│  y = 5           │
└──────────────────┘
```

```
**Step 2: add10(x, y) is called**

Java does NOT send `x` and `y` themselves. It **copies their values** and creates **
```

```
STACK

┌──────────────────┐
│  add10()         │  ← new stack frame
│  x = 4   (copy)  │  ← completely different variable
│  y = 5   (copy)  │  ← completely different variable
├──────────────────┤
│  main()          │
│  x = 4           │  ← original, untouched
│  y = 5           │  ← original, untouched
└──────────────────┘
```

These two xs have the **same name** but they are **completely separate boxes in separate stack frames**. Changing one has zero effect on the other.

```
**Step 3: add10() modifies its local copies**
```

```
STACK

┌──────────────────┐
│  add10()         │
│  x = 14 ✓        │  ← only this x changed
│  y = 15 ✓        │  ← only this y changed
└──────────────────┘
┌──────────────────┐
│  main()          │
│  x = 4           │   ← completely unaffected
│  y = 5           │   ← completely unaffected
└──────────────────┘
```

```
**Step 4: add10() returns, its stack frame is destroyed**
```

```
STACK

┌──────────────────┐
│  main()          │
│  x = 4           │   ← these are the only x,y left
│  y = 5           │
└──────────────────┘
```

When we print again: `4 5`. The copies with 14 and 15 are gone forever.

The Fundamental Principle

> **Call by Value means: the method receives a COPY of the value. Whatever the method does with that copy has no effect on the original.**

This is the ONLY behavior Java has for primitives. No exceptions.

---

4. 🔗 Call by Reference — What It Would Mean (and Why Java Doesn't Have It)

In a language like C++ you can do this:

```
// C++ only — NOT Java
void add10(int &x, int &y) {  // & means "pass the actual variable"
    x = x + 10;  // modifies the ORIGINAL x in main
    y = y + 10;  // modifies the ORIGINAL y in main
}
```

Here `x` in `add10` is not a copy — it IS the same variable as in `main`. Modifying it changes the original.

```
STACK (C++ style — hypothetical)

┌──────────────────────────┐
│  add10()                 │
│  x ──────────────────────┼──┐  ← same variable, just aliased
│  y ──────────────────────┼──┼─ points to main's variables
│                          │  │
│  main()                  │  │
│  x = 4 ◄─────────────────┘  │  ← the real variable
│  y = 5                   │
└──────────────────────────┘
```

Java strictly forbids this. You can never pass a primitive by reference in Java. Period.

---

5. 🎭 The "Illusion" — Objects and Call by Value

Now the interesting part. Look at this code:

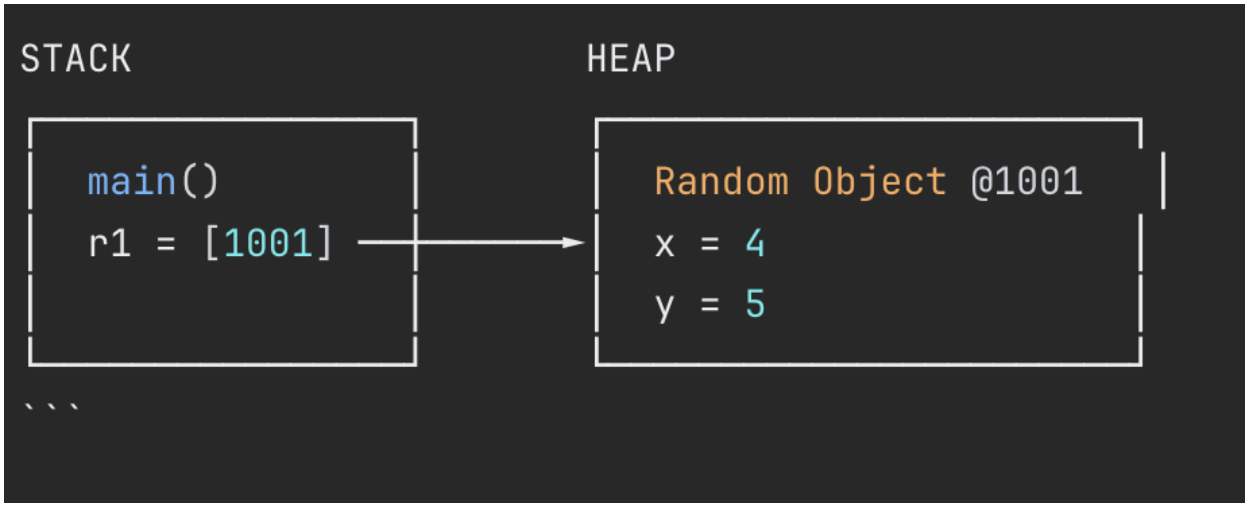
```
class Random {
    int x, y;
    Random(int x, int y) {
        this.x = x;
        this.y = y;
```

```
    }
}

class Demo2 {
    public static void main(String[] args) {
        Random r1 = new Random(4, 5);

        System.out.println(r1.x + " " + r1.y); // 4 5

        add10(r1);

        System.out.println(r1.x + " " + r1.y); // 14 15 !!!
    }

    static void add10(Random r) {
        r.x = r.x + 10;
        r.y = r.y + 10;
    }
}
```

The values DID change this time! How? Java is still call by value!

Complete Memory Walkthrough

Step 1: `new Random(4,5)` **executes**



`r1` stores the **address** (say `1001`) of the object. Not the object itself.

Step 2: `add10(r1)` is called

Java copies the value of `r1` into a new parameter `r`. What is the value of `r1`? It's `1001` — an address.

```
STACK                          HEAP

┌──────────────────┐           ┌──────────────────────────┐
│  add10()         │           │  Random Object @1001     │
│  r = [1001] ─────┼──┐        │  x = 4                   │
├──────────────────┤  └──────► │  y = 5                   │
│  main()          │           └──────────────────────────┘
│  r1 = [1001] ────┼──────────►(same object!)
└──────────────────┘
```

Two different reference variables (`r1` and `r`), but **both contain address `1001`**. Both point to the SAME object.

> **This is still Call by Value!** The VALUE being copied is the address `1001`. Not a reference in the C++ sense.

Step 3: `r.x = r.x + 10` executes inside add10()

`r` holds `1001`, so `r.x` goes to address `1001` and modifies `x` there:

```
STACK                          HEAP

┌──────────────────┐           ┌──────────────────────────┐
│  add10()         │           │  Random Object @1001     │
│  r = [1001] ─────┼──────────►│  x = 14  ← CHANGED!      │
├──────────────────┤           │  y = 15  ← CHANGED!      │
│  main()          │           └──────────────────────────┘
│  r1 = [1001] ────┼──────────►(same object!)
└──────────────────┘
```

The actual object in heap is modified.

Step 4: add10() returns — `r` is destroyed

```
STACK                          HEAP

┌──────────────────┐           ┌──────────────────────────┐
│  main()          │           │  Random Object @1001     │
│  r1 = [1001] ────────────────►│  x = 14                  │
└──────────────────┘           │  y = 15                  │
                               └──────────────────────────┘
```

`r` is gone. But the object at `1001` was permanently modified. `r1` still points to the same address. So printing `r1.x` and `r1.y` gives `14` and `15`.

The Critical Distinction

PRIMITIVE:
main's x = 4 → copy of 4 goes to add10's x
Changing add10's x = 14 has NO effect on main's x

OBJECT REFERENCE:
main's r1 = 1001 → copy of 1001 goes to add10's r
Both r1 and r point to SAME heap object
Changing object through r = changes for everyone

---

6. ❓ Is Java Call by Value or Call by Reference?

The Definitive Answer

Java is ALWAYS and ONLY Call by Value.

Scenario	What gets copied	Effect on original

Passing `int x`	Copy of the integer value	No effect
Passing `Object obj`	Copy of the memory address	Object's fields CAN change, but the reference itself cannot be repointed

The Proof — Why It's NOT Call by Reference

If Java were truly call by reference for objects, this would work:

```
static void tryToReplace(Random r) {
    r = new Random(99, 99); // trying to make r1 point to new object
}

// In main:
Random r1 = new Random(4, 5);
tryToReplace(r1);
System.out.println(r1.x); // What prints?
```

If Java were call by reference: `99` Actual Java output: `4`

```
**Why?**
```

```
STACK                          HEAP

┌──────────────────┐           ┌──────────────────────┐
│ tryToReplace()   │           │ Old Object @1001     │
│ r = [1001] ──────┼──────────►│ x = 4, y = 5         │
└──────────────────┘           └──────────────────────┘
┌──────────────────┐
│ main()           │
│ r1 = [1001] ─────┼──────────► same old object
└──────────────────┘


After r = new Random(99,99):

STACK                          HEAP

┌──────────────────┐           ┌──────────────────────┐
│ tryToReplace()   │           │ Old Object @1001     │
│ r = [2002] ──┐   │           │ x = 4, y = 5         │ ← r1 still here!
└──────────────┼───┘           └──────────────────────┘
┌──────────────┼───┐
│ main()       │   │           ┌──────────────────────┐
│ r1 = [1001] ─┼───┼──────────►│ New Object @2002     │
└──────────────┼───┘           │ x = 99, y = 99       │
               └──────────────►└──────────────────────┘
```

`r` now points to `2002`, but `r1` still points to `1001`. Since `r` is just a local copy of the address, reassigning `r` has no effect on `r1`. This proves Java is call by value.

The Common Interview Mistake

Many freshers say:

- *"Java is call by value for primitives, call by reference for objects"*

This is WRONG. The correct answer:

- **Java is always call by value**
- For objects, the **value being passed is the address/reference**
- This gives a call-by-reference **effect** on the object's fields, but it is NOT true call by reference

---

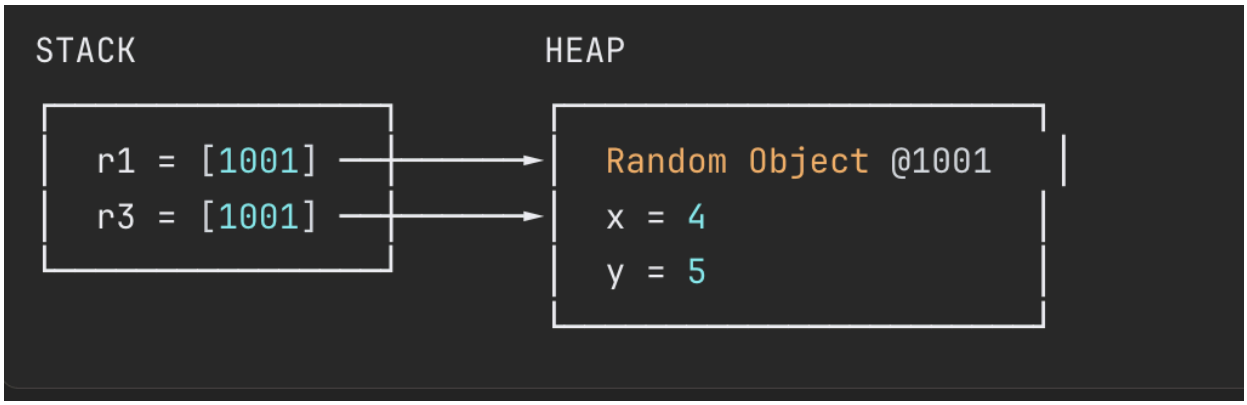
7. 🪞 Shallow Copy vs Deep Copy

These are two ways of "copying" an object. They behave very differently.

Shallow Copy

```
Random r1 = new Random(4, 5);
Random r3 = r1;  // shallow copy
```

What happens:

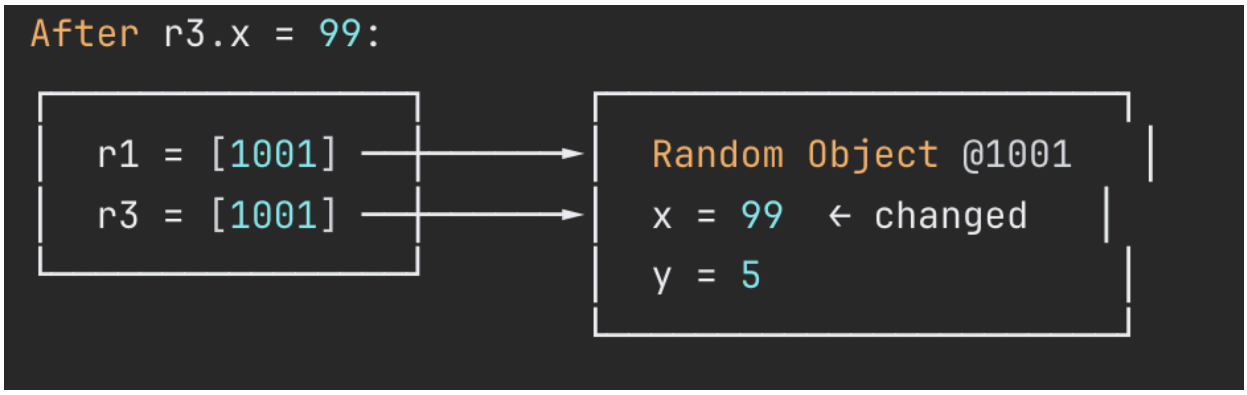


Two reference variables, **one object**. No new object was created. `r1` and `r3` are aliases for the same thing.

Consequence:

```
r3.x = 99;
System.out.println(r1.x); // prints 99 !!
```

Changing through `r3` is visible through `r1` because they ARE the same object.



Deep Copy

Random r1 = new Random(4, 5);
Random r2 = new Random(r1); // special constructor

The special constructor:

```
Random(Random r) {
    this.x = r.x; // copy values
    this.y = r.y; // copy values
}
```

What happens:

```
STACK                          HEAP

┌──────────────────┐
│  r1 = [1001]  ───┼──────────►┌──────────────────────┐
│                  │           │ Random Object @1001  │
│                  │           │ x = 4                │
│                  │           │ y = 5                │
│                  │           └──────────────────────┘
│  r2 = [2002]  ───┼──────────►┌──────────────────────┐
└──────────────────┘           │ Random Object @2002  │
                               │ x = 4  (copied value)│
                               │ y = 5  (copied value)│
                               └──────────────────────┘
```

new keyword creates a **brand new separate object** at a different memory address. Values are the same, but they are **independent copies**.

Consequence:

r2.x = 99;
System.out.println(r1.x); // prints 4 — completely unaffected!

Side-by-Side Comparison

```
### Side-by-Side Comparison
```

```
SHALLOW COPY:                          DEEP COPY:
r1 ─┐                                  r1 ──► [Object A: x=4, y=5]
    ├──► [Object: x=4, y=5]
r3 ─┘                                  r2 ──► [Object B: x=4, y=5]

Change via r3 → r1 sees it             Change via r2 → r1 unaffected
One object, two aliases                Two completely independent objects
No `new` keyword used                  `new` keyword used
```

When to Use Each?

Situation	Use
You want two variables pointing to the same data, any change reflected everywhere	Shallow Copy
You want a true independent duplicate, changes in one don't affect the other	Deep Copy
Caching, multiple views of same object	Shallow
Undo/redo systems, backups, immutable-style operations	Deep

---

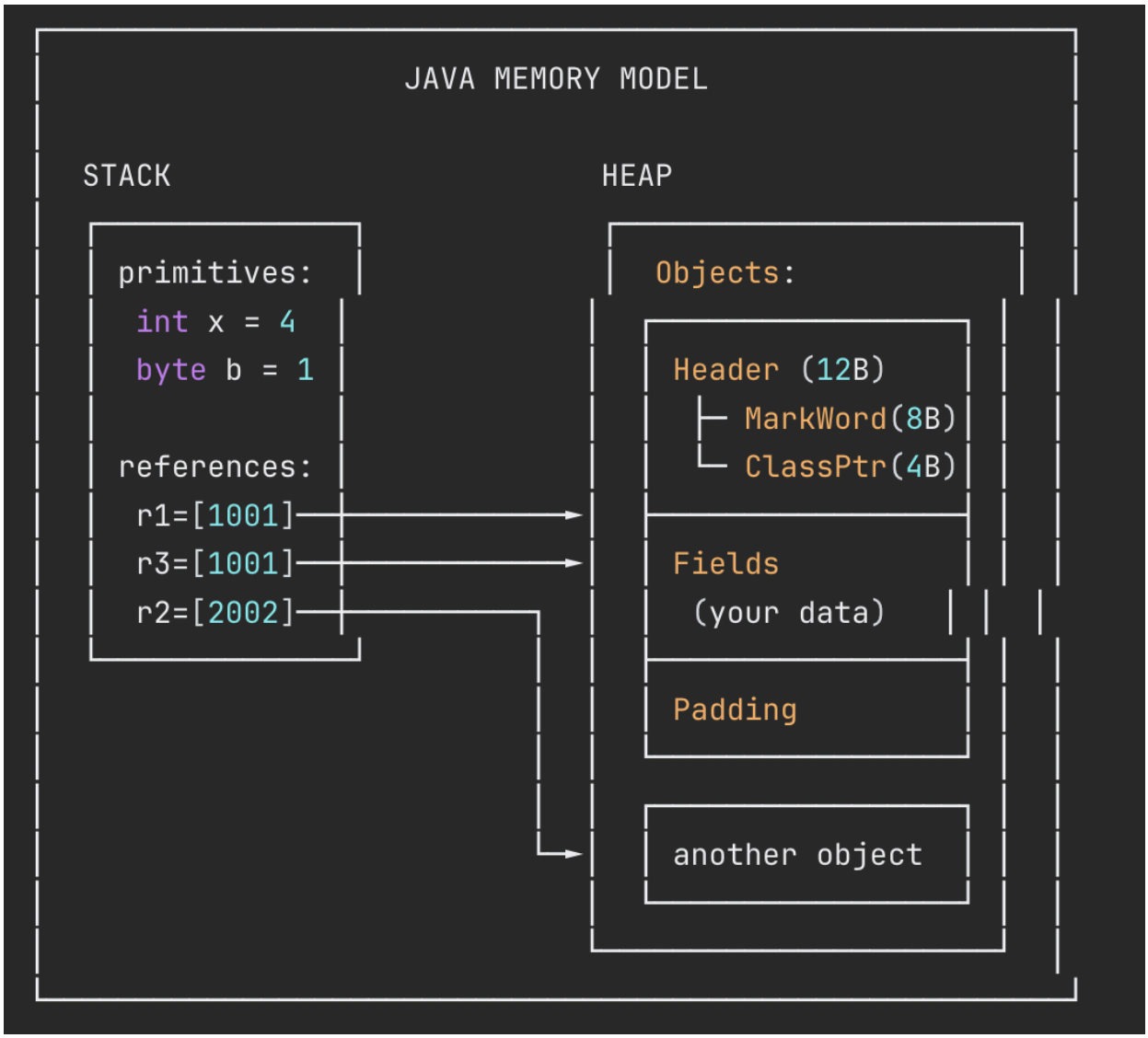
8. 🎁 Bonus: Non-Primitives as Parameters and Return Types

The lecture also mentioned this point:

```
// Non-primitive as parameter
static void add10(Random r) { ... }

// Non-primitive as return type
static Random add10(Random r) {
    r.x += 10;
    r.y += 10;
    return r; // returning the object reference
}
```

Both are perfectly valid. This is fundamental to how Java works — methods can accept and return any type, primitive or non-primitive.

🗺️ Complete Mental Model



Call by Value (ALWAYS in Java):
  Primitive passed → value copied → changes isolated
  Reference passed → address copied → same object modified
                but reference cannot be repointed

Shallow Copy: r3 = r1 → same object, two names
Deep Copy: r2 = new Random(r1) → two separate independent objects

---

Every concept in this lecture is a building block for understanding more advanced Java topics — garbage collection (which uses those GC bits in the mark word), multithreading (which uses those lock bits), and design patterns that rely on deep vs shallow copy semantics. You now have a rock-solid foundation for all of them.

Great catch! Let me find everything I missed.

Going through the transcript carefully...Here are all the concepts that were in the transcript but missing or only briefly touched on in my previous explanation:

Missing Concepts — Complete Deep Explanation

---

1. 🧩 The `this` Keyword Connection to Class Pointer

The transcript specifically mentioned this and I didn't explain it deeply enough.

When we discussed the **Class Pointer** inside the object header, the lecturer directly connected it to the `this` keyword you learned earlier.

What `this` actually is

When you write a class like this:

```
class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;  // what is "this" exactly?
        this.age = age;
    }
}
```

`this` is not magic. It is literally the **same concept as the class pointer** in the object header — it is a hidden reference that every object carries pointing to **itself**.

How it works internally

When you call:

```
Student s1 = new Student("Alice", 20);
```

The JVM secretly passes a hidden argument to the constructor — the address of the newly created object. Inside the constructor, `this` holds that address.

Constructor call (what Java secretly does):

```
Student(this=@1001, name="Alice", age=20) {
   this.name = name;
   // means: go to address 1001, find "name" field, store "Alice" there
}
```

```
HEAP @1001

┌──────────────────────────────┐
│  Header                      │
│   MarkWord       (8 bytes)   │
│   ClassPointer ──────────────► Student.class
│                              │    (this is what "this" refers to)
│  Fields                      │
│   name = "Alice"             │
│   age = 20                   │
└──────────────────────────────┘

        ▲
        │
        └── "this" points here during any method/constructor call
```

So `this.name` means: *"Go to my own object's memory location and access the field called `name`."*

Why every object needs this

Without `this`, when two `Student` objects both call the same method, Java wouldn't know which object's data to use:

```
Student s1 = new Student("Alice", 20);
Student s2 = new Student("Bob", 25);

s1.display(); // Should show Alice's data
s2.display(); // Should show Bob's data
```

The same `display()` method code exists only once in memory (in the class), but `this` gives it the correct object's address each time. The class pointer in the header enables this.

```
## 2. 🔢 Primitive Data Type Sizes — The Foundation

The transcript referenced this as prior knowledge, but it's worth stating completely
```

Data Type	Size	Range
byte	1 byte	-128 to +127
short	2 bytes	-32,768 to +32,767
int	4 bytes	-2^{31} to $2^{31}-1$
long	8 bytes	-2^{63} to $2^{63}-1$
float	4 bytes	~7 decimal digits precision
double	8 bytes	~15 decimal digits precision
char	2 bytes	0 to 65,535 (Unicode)
boolean	1 bit*	true / false
Reference (any object)	4 bytes (or 8B)	Memory address (with comp OOPs)

The lecturer made the point: just as you know `int` takes 4 bytes and `double` takes 8 bytes, **you must also know how much an object takes**. Objects are user-defined data types, and data types must have knowable sizes.

Why byte was specifically used for age

```
class Person {
   byte age;
}
```

The lecturer didn't pick `byte` randomly. A human's age fits within `byte`'s range (-128 to +127), and using `byte` instead of `int` saves 3 bytes per object:

Using int age: Header(12) + int(4) = 16 bytes → padded to 16 ✅
Using byte age: Header(12) + byte(1) = 13 bytes → padded to 16 ✅

In this case the final size is the same due to padding, but the design intention matters — use the smallest type that fits your data.

---

3. 🗑️ Garbage Collection — Introduction via Mark Word

The transcript mentioned garbage collection (GC) in two places and I didn't explain it at all. The lecturer said "we'll read it later" but still introduced it conceptually.

What is Garbage Collection?

In C/C++, you manually free memory:

```
Random* r = new Random();
delete r; // YOU must do this or memory leaks
```

Java handles this automatically. The **Garbage Collector** is a background process in the JVM that automatically finds and deletes objects that are no longer needed.

How does GC know what to delete?

An object is "garbage" when **nothing points to it anymore** — no reference variable holds its address.

```
Random r1 = new Random(4, 5); // object created @1001
r1 = null;                    // r1 no longer points to @1001
// Object @1001 is now unreachable — GC will delete it
```

Connection to Mark Word

The **GC Age** stored in the Mark Word (part of the object header) tells the GC:

```
Mark Word GC fields:
├── Age counter (4 bits)
│   Starts at 0
│   Incremented every time object survives a GC cycle
│   When age reaches 15 (default threshold) →
│      object moves from Young Generation to Old Generation
│
└── GC Mark bits
```

During "Mark" phase of Mark-and-Sweep GC:
  — GC traverses all reachable objects
  — Marks each one as "alive" using these bits
  — Anything NOT marked = garbage = gets swept away

```
### The two memory generations
```

```
HEAP

┌──────────────────────────────────────────────┐
│  Young Generation (Eden + Survivor spaces)   │
│  New objects born here                       │
│  Minor GC runs frequently here               │
│  Objects that survive many GCs get promoted ──► │
├──────────────────────────────────────────────┤
│  Old Generation (Tenured)                    │
│  Long-lived objects live here                │
│  Major GC runs less frequently               │
└──────────────────────────────────────────────┘
```

Connection to the lecture's code

When the lecturer said:

```
static void add10(Random r) {
    r.x += 10;
    r.y += 10;
}
// When this method ends, local "r" variable is destroyed
```

The local `r` reference is destroyed when the method ends. BUT the object in heap is NOT destroyed yet — because `r1` in main still points to it. The GC sees: *"Object @1001 still has r1 pointing to it → keep it alive."*

Only if BOTH `r` and `r1` were gone would the GC consider it collectable.

---

4. 🔒 Locking and Synchronization — Introduction via Mark Word

The transcript mentioned this in context of the Mark Word. Again the lecturer said "we'll cover in multithreading" but introduced it.

Why objects need locking info

In Java, any object can be used as a **lock** (also called a monitor). This is the foundation of thread safety:

```
Random r1 = new Random(4, 5);

synchronized(r1) {    // r1 is being used as a lock!
    // only one thread can be here at a time
    r1.x += 10;
}
```

When thread A enters `synchronized(r1)`, it "locks" the object. Thread B trying to do the same must wait.

How this connects to Mark Word

The Mark Word stores the current lock state of every object:

```
Mark Word Lock States:

Unlocked (normal):
└── Bits indicate: no lock, hashcode stored here

Biased Lock (optimization for single thread):
└── Thread ID stored here
    "This object is almost always used by one thread"
    Faster because no actual lock/unlock needed

Lightweight Lock (two threads competing):
└── Pointer to lock record on stack
    Uses Compare-And-Swap (CAS) operations

Heavyweight/Inflated Lock (high contention):
```

```
└── Pointer to OS-level Monitor object
   Thread must wait in OS queue — expensive!

GC Mark (during garbage collection):
└── GC temporarily overwrites lock bits to mark object
```

This is why the Mark Word must be 8 bytes — it needs to encode all these states plus GC information plus hashcode, all in one compact space.

---

5. 📋 User-Defined Data Types — The Mental Model Shift

The transcript explicitly made this philosophical point which I missed:

Primitives vs User-Defined Types

```
// Primitive — built into Java
int x = 4;        // everyone knows this takes 4 bytes

// User-defined — YOU created this type
Student s1 = new Student();  // how much does THIS take?
```

The lecturer's key insight: **`Student` is just a data type like `int`**. The only difference is:

- `int` was defined by Java creators
- `Student` was defined by YOU

Both are data types. Both must have a definite, calculable size. The lecture teaches you to calculate the size of any user-defined type, just as you always knew the size of primitives.

ALL DATA TYPES IN JAVA	
Primitive (Built-in)	Non-Primitive (User-Defined / Reference)
byte = 1B int = 4B double = 8B etc.	String = Header + fields Student = Header + fields Random = Header + fields MyClass = Header + fields
Value stored directly in stack	Reference (address) stored in stack → object in heap

---

6. 🏗️ Constructor That Takes an Object of Same Type

I completely missed explaining this concept the lecturer demonstrated:

```
class Random {
    int x, y;

    // Constructor 1: takes primitives
    Random(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // Constructor 2: takes an object of SAME type ← this was new
    Random(Random r) {
        this.x = r.x;
        this.y = r.y;
    }
}
```

Why would you want this?

```
Random r1 = new Random(4, 5);
Random r2 = new Random(r1);   // uses Constructor 2
```

You want `r2` to start with the same values as `r1`, but be completely **independent**. This is the **Copy Constructor** pattern.

Memory picture

Step 1: Random r1 = new Random(4, 5)

```
STACK         HEAP
r1 = @1001 ──► [@1001: x=4, y=5]
```

Step 2: Random r2 = new Random(r1)

```
      ↑
      "new" = allocate NEW space in heap
```

```
STACK         HEAP
r1 = @1001 ──► [@1001: x=4, y=5]  ← original
r2 = @2002 ──► [@2002: x=4, y=5]  ← new independent copy
```

Inside Constructor 2, what happens:

```
Random(Random r) {
   // "r" receives a COPY of r1's address (@1001)
   // so r also points to @1001 temporarily

   this.x = r.x;
   // this = @2002 (the new object being built)
   // r.x  = go to @1001, get x = 4
   // this.x = store 4 at @2002's x field

   this.y = r.y;
   // same logic, stores 5 at @2002's y field
}
```

After construction, `r2` has its own memory at `@2002` with copied values. They are fully independent.

This IS the deep copy mechanism

```
// This creates a deep copy:
Random r2 = new Random(r1);  // new object, copied values

// This creates a shallow copy:
Random r3 = r1;  // NO new keyword, just copying the address
```

The copy constructor is the standard Java pattern for intentional deep copying.

---

7. 🔄 The new Keyword — Its Exact Role

The transcript repeatedly referred to `new` and the lecturer made specific points about it that I didn't compile together:

Every time you write new:

```
new Random(4, 5)
 │
 ├── 1. Calculate total size needed for the object
 │      (Header + Fields + Padding)
 │
 ├── 2. Request that much space from Heap
 │
 ├── 3. JVM allocates contiguous bytes in Heap
 │      Returns the starting address (e.g., @1001)
 │
 ├── 4. Zero-initialize all fields
 │      (int fields become 0, references become null)
 │
 ├── 5. Write the object header
 │      (MarkWord initialized, ClassPointer set)
 │
 ├── 6. Call the constructor with "this" = @1001
 │      (constructor fills in your actual values)
 │
 └── 7. Return the address @1001
       stored in your reference variable
```

No new = no new object

```
Random r3 = r1;       // NO new = NO new heap allocation
```

```
                    // r3 just gets a copy of r1's address
                    // both point to SAME object

Random r2 = new Random(r1); // HAS new = NEW heap allocation
                            // r2 gets a fresh address
                            // independent object
```

This is the single most important distinction for understanding shallow vs deep copy.

---

8. 🎯 Non-Primitives as Method Parameters AND Return Types

The lecturer specifically called this out as important. I mentioned it briefly but didn't explain it fully.

Non-primitive as parameter

```
static void add10(Random r) {
    r.x += 10;
    r.y += 10;
}
```

`r` here is a reference variable. It's a parameter just like `int x` would be — but instead of holding a number, it holds an address. Everything we discussed about call by value applies.

Non-primitive as return type

```
static Random add10(Random r) {
    r.x += 10;
    r.y += 10;
    return r;  // returning the reference (address)
}
```

When you return `r`, you're returning the **address** of the object. The caller receives that address:

```
Random r1 = new Random(4, 5);
Random result = add10(r1);

// What is result?
// result holds the SAME address as r1
```

```
// because add10 returned r, which held r1's address

System.out.println(result == r1); // true! same object
```

You can return a new object too

```
static Random createDoubled(Random r) {
    return new Random(r.x * 2, r.y * 2);
    // creates and returns a brand new object
}

Random r1 = new Random(4, 5);
Random r2 = createDoubled(r1);
// r1 → @1001 (x=4, y=5)  — unchanged
// r2 → @2002 (x=8, y=10) — new independent object
```

---

9. 🌐 64-bit JVM — Why It Matters for Reference Size

The transcript explained the 4 vs 8 byte reference distinction more carefully than I captured:

The full explanation

```
32-bit JVM:
├── Can address 2³² = 4GB of memory maximum
├── Pointers are 32 bits = 4 bytes
└── References = 4 bytes (naturally)

64-bit JVM (default today):
├── Can theoretically address 2⁶⁴ = 18 exabytes
├── But most programs don't need anywhere near that
├── Pointers would be 64 bits = 8 bytes (wasteful!)
│
├── Solution: Compressed Ordinary Object Pointers (OOPs)
│   ├── Heap up to 32GB: use 4-byte compressed pointers
│   │   JVM scales addresses by 8 (since objects are
│   │   8-byte aligned anyway)
│   │   4 bytes × 8 = 32-byte range per pointer
│   │   = 4GB × 8 = 32GB addressable ✓
│   │
│   └── Heap > 32GB: must use full 8-byte pointers
│
└── Default: 4 bytes (compressed OOPs enabled by default)
```

Override: -XX:-UseCompressedOops (forces 8 bytes)

Practical impact on object size

With Compressed OOPs (default, heap < 32GB):
Student object:

```
MarkWord     = 8 bytes
ClassPointer = 4 bytes  ← compressed
name ref     = 4 bytes  ← compressed
age          = 4 bytes
rollNo       = 4 bytes
college ref  = 4 bytes  ← compressed
Total fields = 16 bytes
Header       = 12 bytes
Padding      = 4 bytes
TOTAL        = 32 bytes
```

Without Compressed OOPs (heap > 32GB):

```
MarkWord     = 8 bytes
ClassPointer = 8 bytes  ← full 64-bit
name ref     = 8 bytes  ← full 64-bit
age          = 4 bytes
rollNo       = 4 bytes
college ref  = 8 bytes  ← full 64-bit
Total fields = 24 bytes
Header       = 16 bytes
Padding      = 0 bytes  (16+24=40, already multiple of 8)
TOTAL        = 40 bytes
```

This is why the lecturer said "4 bytes or 8 bytes depending on JVM" — it's a real, calculable difference.

---

10. 💡 Why C++ Has Call by Reference but Java Doesn't — The Design Decision

The transcript makes a passing but important point about C++ vs Java design philosophy that I didn't explain:

C++ allows true call by reference

// C++

```
void add10(int &x, int &y) {  // & = pass by reference
    x += 10;  // modifies caller's actual variable
    y += 10;
}

int main() {
    int x = 4, y = 5;
    add10(x, y);
    cout << x << y;  // prints 14 15
}
```

Why Java removed this

Java's designers made a deliberate choice to eliminate call by reference for **safety and simplicity**:

```
Problems with call by reference:
├── Hard to reason about: calling f(x) might change x
│   You must read f's implementation to be sure
│
├── Aliasing bugs: multiple references to same variable
│   cause subtle, hard-to-find bugs
│
├── Violates encapsulation: functions can secretly
│   modify caller's state
│
└── Memory safety: in C++, references to local variables
    can outlive those variables → undefined behavior
```

Java's decision:

> *"Every method gets its own copy of every argument. Methods cannot secretly modify the caller's primitive variables. Period."*

This makes Java programs easier to reason about — when you call a method, your local primitives are safe.

The tradeoff: You can't directly modify a caller's primitive from a method. If you need that behavior, you must wrap the value in an object (like the `Random` class example in the lecture).

🗺️ Complete Picture — Everything Together

JAVA OBJECTS — COMPLETE MENTAL MODEL

```
1. REFERENCE SIZE (in Stack)
   └── 4 bytes (compressed OOPs, default)
   └── 8 bytes (no compression, large heaps)

2. OBJECT SIZE (in Heap) = Header + Fields + Padding
   ├── Header
   │   ├── Mark Word (8 bytes)
   │   │   ├── Locking info (biased/lightweight/heavyweight)
   │   │   ├── GC age counter
   │   │   ├── GC mark bits
   │   │   └── Identity hashcode
   │   └── Class Pointer (4 bytes)
   │       └── Points to Class definition
   │           (enables "this", instanceof, polymorphism)
   ├── Fields (your declared variables)
   │   └── Each sized by its type
   └── Padding (0 to 7 bytes)
       └── Makes total a multiple of 8

3. CALL BY VALUE (Java's ONLY mechanism)
   ├── Primitive: copies the value
   │   └── Method changes are isolated — caller unaffected
   └── Reference: copies the address
       └── Both caller and method address SAME heap object
           └── Field changes ARE visible to caller
               BUT reassigning the reference is NOT

4. COPY TYPES
   ├── Shallow Copy: Random r3 = r1
   │   └── Two references → ONE object
   │       Change via either = change visible from both
   └── Deep Copy: Random r2 = new Random(r1)
       └── Two references → TWO separate objects
           Change via one = invisible to the other

5. JAVA IS PURELY CALL BY VALUE
   ├── Primitives: obvious call by value
   └── Objects: still call by value (address is copied)
       └── LOOKS like call by reference for field changes
```

but is NOT (can't repoint caller's reference)

Every single concept from the transcript is now covered. The ones I missed were subtle but critical — especially the `this`/class pointer connection, the copy constructor pattern, the `new` keyword's exact role, and the Java vs C++ design philosophy.

In the video I mentioned:

Random r3 = r1;

as shallow copy.

Technically this is known as reference copying (aliasing).
It is often loosely called shallow copy for beginners.

We will cover Reference Copying and more in-depth on Shallow vs Deep Copy in an upcoming video of Immutable Objects once we master basics of OOPS.